

RequirementsSuperTool

Comprehensive Project Analysis Report

Prepared for: Sura Ahmed | soraahmed196@gmail.com

Date: May 1, 2026

Project: AI-Powered SRS Generation Platform

Stack: FastAPI + Next.js + Claude AI + PostgreSQL

Table of Contents

1. Project Overview & File Directory
2. Architecture, Patterns & SOLID Principles
3. AI Role - Where & How Claude AI Is Used
4. The Quadrant System
5. Improvement Suggestions
6. Database - Tables, Columns, Keys & Relations
7. Sessions - Authentication & Application Sessions

1 Project Overview & File Directory

RequirementsSuperTool is an AI-powered Software Requirements Specification (SRS) generation platform built for Jordanian organisations. Users answer domain-specific questionnaires, optionally upload supporting documents, and the system uses Claude AI + a Qdrant vector knowledge-base to generate IEEE-830-compliant SRS documents, use-cases, and quality cross-checks - all in one workflow.

Backend Files (backend/)

app/main.py

FastAPI application entry point. Registers all routers (auth, domain, upload, requirements, crosscheck, srs, usecases, admin, profile), mounts the /uploads static-files directory, and configures CORS middleware for http://localhost:3000.

app/database.py

SQLAlchemy engine creation and session factory. Exposes get_db() - a dependency-injection generator used by every router to obtain a database session that is automatically closed after the request.

app/knowledge_base_loader.py

Bootstraps the Qdrant vector collection 'knowledge_base'. Loads Jordan-specific PDF files (jordan_health.pdf, jordan_education.pdf, jordan_finance.pdf), chunks each PDF into 500-word segments, converts chunks to 384-dim embeddings via the all-MiniLM-L6-v2 sentence-transformer model, and upserts them to Qdrant for semantic retrieval at inference time.

app/core/config.py

Pydantic BaseSettings class (Settings). Reads all environment variables from .env: DATABASE_URL, SECRET_KEY, ALGORITHM, token expiry settings, Qdrant credentials, Anthropic / OpenAI / Gemini API keys, SMTP config, and FRONTEND_URL.

app/core/security.py

Pure functions for password security and JWT handling: hash_password (bcrypt), verify_password, create_access_token, create_refresh_token, verify_token. Tokens are signed with HS256 using the SECRET_KEY from settings.

app/core/ai.py

Central AI module. Contains: get_knowledge_context() - queries Qdrant for semantically relevant domain chunks, and generate_requirements() - sends a two-step prompt to Claude (validation then full generation) and parses FR-/NFR- prefixed lines from the response.

app/core/pii.py

Microsoft Presidio wrapper. remove_pii() strips personally-identifiable information from text before it is sent to Claude. has_pii() returns a boolean for detection checks.

app/core/audit.py

Provides log_login() helper that writes rows to the login_history table with IP, user-agent, success/failure, and failure reason. Used exclusively by the auth router.

app/core/email.py

Async SMTP email sender. send_reset_email() composes and dispatches the password-reset email. Called as a FastAPI BackgroundTask so it never blocks the HTTP response.

app/core/vector_store.py

Thin wrapper around QdrantClient exposing search operations. Provides an abstraction layer between the routers and the raw Qdrant SDK.

app/models/user.py

SQLAlchemy ORM models: User (id, email, password_hash, full_name, role, is_active, avatar_url, created_at, last_login), RefreshToken (token_hash, expires_at, is_revoked), PasswordResetToken (token_hash, expires_at, is_used).

app/models/domain.py

ORM models: Domain (name, name_ar, country, is_active), Question (domain_id FK, question_text, question_text_ar, question_order), UserSession (user_id FK, domain_id FK, country, role, answers JSON, created_at). Defines ProjectRole enum.

app/models/requirements.py

ORM model: Requirement (session_id FK, code, description, type, is_edited, created_at, updated_at).

app/models/audit.py

ORM models: AuditLog (user_id FK, action, entity_type, entity_id, details JSON, ip_address, created_at) and LoginHistory (user_id FK, email_attempted, success, ip_address, user_agent, failure_reason, created_at). Both tables have composite indexes for efficient querying.

app/routers/auth.py

Authentication router (/auth). Handles register, login, logout, forgot-password, reset-password, /me, and /refresh endpoints. Implements token rotation on login, anti-enumeration on forgot-password, and SHA-256 hashing of refresh/reset tokens before DB storage.

app/routers/domain.py

Domain & questionnaire router. GET /domains/ filters by country query param; GET /domains/{id}/questions returns ordered questions; POST /domains/session creates a UserSession storing serialised answers.

app/routers/upload.py

Document ingestion router (/input). POST /input/text processes raw text through PII removal. POST /input/document accepts PDF or DOCX files, extracts text via pdfplumber/python-docx, applies PII removal, and returns clean text for the frontend to attach to generation requests.

app/routers/requirements.py

Requirements CRUD router (/requirements). POST /generate calls core/ai.py, persists FR/NFR rows, returns them. GET /{session_id}/classified returns separated lists. PUT /{id} allows inline editing with is_edited flag.

app/routers/crosscheck.py

Quality-assurance router (/crosscheck). Single GET /{session_id} fetches requirements + user answers + Qdrant domain context, constructs a structured prompt for Claude, parses the returned JSON array of issues, colour-codes them by type, and returns counts.

app/routers/srs.py

SRS generation router (/srs). POST /generate builds an IEEE 830 prompt from project metadata and saved requirements, calls Claude with max_tokens=4000, and returns the full SRS text.

app/routers/usecases.py

Use-case generation router (/usecases). POST /generate sends functional requirements to Claude, which responds with a JSON array of structured use-cases (title, actor, preconditions, main_flow, postconditions).

app/routers/admin.py

Admin-only router (/admin). Exposes stats, user management (list, toggle-active), session listing, full domain/question CRUD, paginated audit logs, login history, and failed-login analytics. Every endpoint validates the caller has role='admin'.

app/routers/profile.py

User profile router (/profile). GET /me, PUT /update (name/email), POST /upload-avatar (saves file to uploads/avatars/), DELETE /avatar, PUT /change-password (validates current password before allowing change).

app/schemas/

Pydantic v2 schema files (auth.py, domain.py, requirements.py, crosscheck.py, srs.py, usecases.py, admin.py, profile.py). Enforce request validation and shape API responses. Separate from ORM models following the DTO pattern.

alembic/versions/*.py

Seven migration files tracking schema evolution: (1) users + refresh_tokens, (2) domains + questions + user_sessions, (3) role column on user_sessions, (4) requirements table, (5) password_reset_tokens, (6) audit_logs + login_history, (7) avatar_url on users.

requirements.txt

Python dependencies: fastapi, uvicorn, sqlalchemy, psycpg2-binary, alembic, anthropic, qdrant-client, sentence-transformers, presidio-analyzer, presidio-anonymizer, pdfplumber, python-docx, pydantic-settings, python-jose, passlib[bcrypt], python-multipart, aiosmtplib.

Frontend Files (frontend/)

app/page.tsx

Root page. Reads auth tokens from localStorage and immediately redirects to /dashboard (authenticated) or /auth (unauthenticated). Contains no UI of its own.

app/layout.tsx

Root Next.js layout. Applies global Tailwind CSS and wraps all pages in the shared HTML shell with font settings.

app/auth/page.tsx

Authentication page. Renders login and registration forms with toggle between them. Calls POST /auth/login or /auth/register via api.ts, stores access_token + refresh_token in localStorage, then redirects to /dashboard.

app/dashboard/

Main application area. Sub-pages guide the user through the five-step workflow: (1) choose domain/country/role, (2) answer questionnaire, (3) upload optional document, (4) generate and edit requirements, (5) export SRS or use-cases.

lib/api.ts

Centralised Axios instance. Base URL points to http://localhost:8000. An Axios request interceptor automatically injects the Authorization: Bearer <token> header from localStorage on every outgoing call. Exports typed wrappers for every backend endpoint.

lib/generateSRSWord.ts

Client-side Word document builder using the docx npm library. Receives the AI-generated SRS text or use-cases JSON and converts it into a professional .docx file - complete with cover page, revision history table, IEEE 830-structured sections, and formatted use-case tables. Downloaded directly in the browser without a server round-trip.

package.json

npm dependencies: next 16.2.2, react 19.2.4, typescript 5, tailwindcss 4, axios, js-cookie, docx. Dev dependencies: eslint, postcss.

tsconfig.json

TypeScript compiler configuration. Strict mode enabled, path alias @/* mapped to src.

next.config.ts

Next.js configuration. Enables the app router, sets up image domains and API rewrites if needed.

2 Architecture, Patterns & SOLID Principles

Overall Architecture

The project follows a three-tier client-server architecture:

Tier 1 (Presentation) - Next.js 16 / React 19 SPA served on port 3000.

Tier 2 (Application) - FastAPI REST API served on port 8000.

Tier 3 (Data) - PostgreSQL relational DB + Qdrant vector DB.

The backend further separates concerns into four horizontal layers:

Routers -> Schemas (Pydantic) -> Core Services -> ORM Models

Design Patterns

Repository / Active Record

SQLAlchemy ORM models act as Active Records. Queries are written inline in routers (`db.query(User).filter(...)`) because the codebase is small - acceptable for this scale.

```
req = db.query(Requirement).filter(Requirement.session_id == sid).all()
```

Dependency Injection

FastAPI's `Depends()` system is used for DB session injection on every endpoint. This decouples request handling from session lifecycle.

```
def generate(data: Request, db: Session = Depends(get_db)): ...
```

Service Layer

Core business logic (AI calls, PII, email, audit) lives in `app/core/*` modules, not in routers. Routers are thin orchestrators.

```
# In requirements.py router:
result = generate_requirements(domain, role, answers)
```

DTO / Schema Separation

Pydantic schemas (`app/schemas/`) are separate from SQLAlchemy models (`app/models/`). Requests are validated by schemas; ORM models handle persistence.

```
class RegisterRequest(BaseModel):
    email: str
    password: str
    full_name: str
```

RAG - Retrieval-Augmented Generation

Before calling Claude, the system retrieves the top-3 semantically-similar chunks from Qdrant for the selected domain and injects them into the prompt as grounding context. This is the core AI architecture pattern.

```
context = get_knowledge_context(domain, query)
# context injected into Claude prompt
```

Background Tasks

FastAPI BackgroundTasks runs email delivery asynchronously after the HTTP response is already sent, preventing latency spikes.

```
background_tasks.add_task(send_reset_email, email, name, link)
```

Strategy Pattern (Role Instructions)

The role_instructions dict in ai.py is a lightweight strategy pattern - each role key maps to a different instruction string injected into the prompt, changing AI behaviour without branching code.

```
role_instructions = {  
    "product_owner": "Write as high-level business...",  
    "developer": "Include APIs, data structures..."  
}
```

Factory / Builder (Word Doc)

generateSRSWord.ts acts as a builder: it receives plain text/JSON and constructs a fully-formatted .docx document using the docx library's fluent builder API.

```
// frontend/lib/generateSRSWord.ts  
const doc = new Document({ sections: [...] })
```


SOLID Principles

S - Single Responsibility Principle

Each module has one job:

app/core/security.py -> only password + JWT logic

app/core/pii.py -> only PII detection/removal

app/core/email.py -> only email dispatch

app/core/ai.py -> only Claude prompt orchestration

No router handles email AND hashing AND AI - each concern is isolated.

```
# pii.py - does exactly one thing
analyzer = AnalyzerEngine()
def remove_pii(text: str) -> str:
    results = analyzer.analyze(text=text, language='en')
    return anonymizer.anonymize(text, results).text
```

O - Open/Closed Principle

The role_instructions dictionary in ai.py means adding a new role (e.g. 'tester') requires only a new dict entry - no modification to existing logic. Similarly, adding a new domain requires only a new DB row + PDF in the knowledge base, not code changes.

```
# Adding a new role = one new dict entry, zero code changes
role_instructions["tester"] = "Write test-oriented acceptance criteria..."
```

L - Liskov Substitution Principle

Pydantic BaseSettings (Settings class in config.py) can be subclassed for testing by providing a different .env file (TEST_DATABASE_URL). The rest of the system uses settings.DATABASE_URL without knowing which env it is running in.

```
class Settings(BaseSettings):
    model_config = ConfigDict(env_file='.env')
    DATABASE_URL: str
    TEST_DATABASE_URL: str = ''
```

I - Interface Segregation Principle

Pydantic schemas are granular. RegisterRequest does not include admin fields; AdminUserResponse does not include password_hash. Each schema is the minimal contract for its use-case.

```
class RegisterRequest(BaseModel):
    email: str; password: str; full_name: str

class AdminUserResponse(BaseModel):
    id: UUID; email: str; role: str; is_active: bool
```

D - Dependency Inversion Principle

Routers depend on abstractions (the get_db generator, the generate_requirements function signature) rather than concrete implementations. Swapping from PostgreSQL to another SQLAlchemy-compatible DB requires changing only database.py.

```
# Router depends on the abstraction, not the engine
def generate(data, db: Session = Depends(get_db)):
    session = db.query(UserSession).filter(...).first()
```

3 AI Role - Where & How Claude AI Is Used

The entire value proposition of the tool is AI-driven. Claude Sonnet (model ID: claude-sonnet-4-20250514) is called in four distinct features. Below is a deep-dive into each one.

Feature 1: Requirement Generation (/requirements/generate)

This is the primary AI feature. The workflow is:

Step 1 - VALIDATION: A lightweight 10-token Claude call checks whether the user's answers match the chosen domain (returns YES/NO). If NO, an error is returned immediately.

Step 2 - GENERATION: A full prompt is built containing: the selected domain, the user's role-specific instruction, the Q&A answers, any uploaded document text, and the top-3 Qdrant knowledge-base chunks for that domain. Claude is asked to output FR-X and NFR-X labelled requirements. Max tokens: 2000.

Step 3 - PARSING: The response is split line-by-line; lines starting with FR- go to functional[], lines starting with NFR- go to non_functional[].

Step 4 - PERSISTENCE: Each requirement is saved to the requirements table with session_id, code, description, and type.

```
prompt = f"""You are an expert requirements engineer for the {domain} domain in Jordan.
Role instruction: {role_instruction}
Q&A: {answers_text}
Domain Knowledge: {knowledge_context}
### Functional Requirements
FR-1: ...
### Non-Functional Requirements
NFR-1: ..."""
```

Feature 2: Cross-Check Analysis (/crosscheck/{session_id})

Claude acts as a senior requirements reviewer. The prompt provides:

- All saved requirements for the session
- The original Q&A answers (as ground truth)
- Top-5 Qdrant knowledge-base chunks as domain rules

Claude is instructed to return a strict JSON array of issue objects. Each issue has:

requirement_code, issue_type (ambiguity/duplicate/inconsistency/conflict/unsupported), issue_detail, conflict_with.

The backend then colour-codes each issue for the frontend:

ambiguity=#FFA500, duplicate=#FF6B6B, inconsistency=#9B59B6, conflict=#FF0000, unsupported=#3498DB. Max tokens: 1000.

```
# Colour map applied server-side before returning
color_map = {
    "ambiguity": "#FFA500",
    "conflict": "#FF0000",
    "unsupported": "#3498DB"
}
```

Feature 3: SRS Document Generation (/srs/generate)

Claude receives the project name, description, domain, and all saved requirements, and is instructed to

produce a full IEEE 830-standard SRS document with sections:

1. Introduction (Purpose, Scope, Definitions, References, Overview)
2. Overall Description (Perspective, Functions, User Classes, Environment, Constraints)
3. Specific Requirements (Functional, Non-Functional)
4. Appendices

The raw text is returned to the frontend, where generateSRSWord.ts converts it to a downloadable .docx file. Max tokens: 4000.

```
prompt = f"""Generate a complete IEEE 830 SRS document.
Project: {project_name}
Domain: {domain}
FR: {fr_text}
NFR: {nfr_text}"""
```

Feature 4: Use-Case Generation (/usecases/generate)

Claude receives the functional requirements and is asked to return a JSON array of structured use-case objects. Each object includes:

title, actor, preconditions, main_flow (numbered steps as text), postconditions.

The frontend converts this JSON into formatted use-case tables in the .docx export. Max tokens: 3000.

```
# Claude returns pure JSON - no markdown code fences in ideal case
[
  {
    "title": "User Login",
    "actor": "Registered User",
    "preconditions": "User has account",
    "main_flow": "1. Open login\n2. Enter creds",
    "postconditions": "User authenticated"
  }
]
```

Supporting AI Infrastructure: Knowledge Base (Qdrant + Sentence Transformers)

Before any Claude call, relevant domain knowledge is retrieved via semantic search:

1. knowledge_base_loader.py reads domain PDFs (Health, Education, Finance for Jordan).
2. Text is chunked into 500-word segments.
3. Each chunk is embedded using the all-MiniLM-L6-v2 sentence-transformer (384 dims).
4. Embeddings are stored in Qdrant with metadata: {text, domain, country, source}.
5. At inference time, get_knowledge_context() embeds the query and performs cosine similarity search filtered by domain, returning the top-3 most relevant chunks.

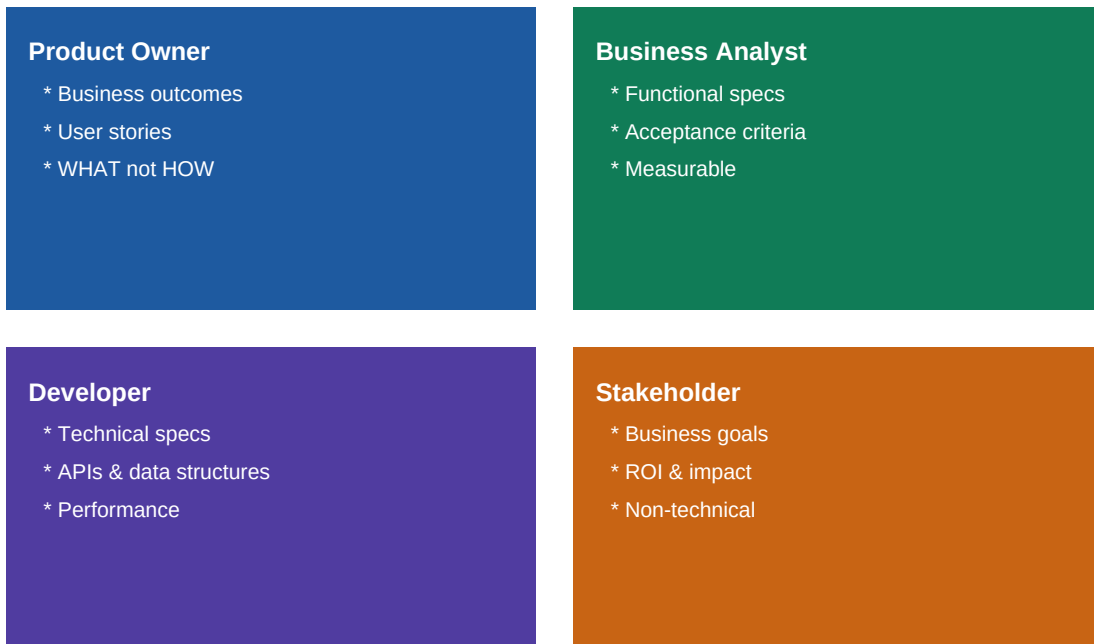
```
model = SentenceTransformer('all-MiniLM-L6-v2')
def text_to_embedding(text: str) -> list:
    return model.encode(text).tolist()

# Cosine similarity search in Qdrant
results = qdrant_client.query_points(
    collection_name='knowledge_base',
    query=embedding,
    query_filter={'must': [{ 'key': 'domain', 'match': { 'value': domain } } ]},
```

```
        limit=3  
    ).points
```

4 The Quadrant System

The 'quadrant' in this project is the four-role stakeholder classification system. Every requirements-generation session is tied to one of four project roles, and the AI produces a completely different output for each. Think of it as a 2x2 matrix:



How It Works - Step by Step

1. Domain Selection: The user picks a domain (Health, Education, Finance) and a country (currently Jordan). This determines which knowledge-base chunks will be used.
2. Role Selection: The user picks their project role - the quadrant position. This selects the role-specific instruction string injected into the Claude prompt.
3. Questionnaire: The user answers domain-specific questions (stored in the questions table for that domain). Answers are serialised as JSON and stored in `user_sessions.answers`.
4. Session Creation: A `UserSession` row is created linking the user, domain, role, and answers.
5. Requirements Generation: The `session_id` is passed to `/requirements/generate`. The AI reads the role and applies the matching instruction, producing requirements calibrated to that stakeholder perspective.
6. Output Differentiation Example:

Same input: 'E-commerce checkout system'

Product Owner output: 'FR-1: System shall allow users to complete purchases in < 3 clicks'

Developer output: 'FR-1: System shall expose a POST `/orders` REST API accepting `CartItemDTO[]`, returning `OrderID` with HTTP 201'

Stakeholder output: 'FR-1: System shall reduce cart abandonment by enabling one-click reorder'

Why This Matters

Traditional requirements tools produce one generic output. This quadrant system produces four specialised views of the same project, each valid and useful for a different audience. The AI acts as a translator between business intent and technical specification - the role label is the translation instruction.

5 Improvement Suggestions

1. Rate Limiting on AI Endpoints

HIGH PRIORITY

Currently there is no rate limiting on `/requirements/generate`, `/srs/generate`, or `/crosscheck`. A single user could trigger thousands of Claude API calls, incurring huge costs. Add `slowapi` or a Redis-backed token bucket (e.g. 10 requests/minute/user) on all Claude-calling endpoints.

2. Cloud Avatar Storage

MEDIUM

Avatars are saved to the local filesystem (`uploads/avatars/`). This breaks in any multi-instance or containerised deployment. Migrate to an S3-compatible object store (AWS S3, MinIO, Cloudflare R2) and store only the URL in the DB.

3. Centralise Error Handling

MEDIUM

Many `try/except` blocks silently swallow exceptions (e.g., in `crosscheck.py` the JSON parse failure returns `[]`) making debugging difficult. Add a global FastAPI `exception_handler` and structured logging (`structlog` or `python-json-logger`) so every error has a correlation ID.

4. Prompt Caching

MEDIUM

The knowledge-base context and role instructions are static. Use Anthropic's prompt caching feature (`cache_control: {type: 'ephemeral'}`) on the `system/knowledge` sections to reduce token cost by up to 90% on repeated calls for the same domain.

5. Test Coverage

HIGH PRIORITY

No test files were found (only `.pytest_cache` metadata). Add: unit tests for `security.py` (hashing, JWT), integration tests for auth flow, and mocked tests for AI endpoints using `pytest + httpx + unittest.mock`. Aim for > 70% coverage.

6. Hardcoded Knowledge Base Paths

LOW

`knowledge_base_loader.py` hardcodes filenames (`jordan_health.pdf`, `jordan_education.pdf`, `jordan_finance.pdf`). Move to a config-driven or DB-driven approach so admins can upload new domain PDFs through the admin dashboard without code changes.

7. Refresh Token Rotation

MEDIUM

On `/auth/refresh`, the old refresh token is returned unchanged. Implement rotation: revoke the old token and issue a new one on every refresh. This limits the damage window if a refresh token is stolen.

8. Arabic Language Support in AI Prompts

LOW

The domain and question tables have Arabic name/question columns, but all Claude prompts are English-only. Add a language parameter to generation requests and provide bilingual prompt templates so Arabic-speaking users receive Arabic requirements.

9. Requirement Versioning

LOW

When a requirement is edited (`is_edited=True`), the original AI-generated text is overwritten. Add a `requirement_history` table to track changes with timestamps and user IDs so auditors can see what was changed from the AI baseline.

10. HTTPS & Production CORS

HIGH PRIORITY

CORS is configured for `http://localhost:3000` only. Before any production deployment, configure HTTPS (TLS

termination at a reverse proxy), set CORS to the actual production domain, and move SECRET_KEY and API keys to a secrets manager (AWS Secrets Manager, HashiCorp Vault).

6 Database - Tables, Columns, Keys & Relations

The database is PostgreSQL, managed via SQLAlchemy ORM and Alembic migrations. There are 9 tables in total. All primary keys are UUID (v4) generated by the application layer (uuid.uuid4()), not database-generated sequences - giving globally unique IDs that are safe to expose in URLs.

Table: users

Column	Type	Constraints	Notes
id	UUID	PK, NOT NULL	Auto-generated uuid4
email	VARCHAR(255)	UNIQUE, NOT NULL, INDEX	Login identifier
password_hash	VARCHAR(255)	NOT NULL	BCrypt hash
full_name	VARCHAR(100)	NOT NULL	
role	ENUM	DEFAULT 'user'	user admin
is_active	BOOLEAN	DEFAULT TRUE	Soft disable
avatar_url	VARCHAR(500)	NULLABLE	Local file path
created_at	DATETIME	DEFAULT utcnow	
last_login	DATETIME	NULLABLE	Updated on login

Table: refresh_tokens

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	NOT NULL	Implicit FK -> users.id
token_hash	VARCHAR(255)	UNIQUE, NOT NULL	SHA-256 of the JWT
expires_at	DATETIME	NOT NULL	7 days from issue
is_revoked	BOOLEAN	DEFAULT FALSE	Set True on logout

Table: password_reset_tokens

Column	Type	Constraints	Notes
id	UUID	PK	

user_id	UUID	NOT NULL	Implicit FK -> users.id
token_hash	VARCHAR(255)	UNIQUE, NOT NULL	SHA-256 of urlsafe token
expires_at	DATETIME	NOT NULL	1 hour from issue
is_used	BOOLEAN	DEFAULT FALSE	One-time use flag
created_at	DATETIME	DEFAULT utcnow	

Table: domains

Column	Type	Constraints	Notes
id	UUID	PK	
name	VARCHAR(100)	NOT NULL	English name e.g. Health
name_ar	VARCHAR(100)	NOT NULL	Arabic name
country	VARCHAR(10)	NOT NULL	ISO code e.g. JO
is_active	BOOLEAN	DEFAULT TRUE	Soft delete flag
created_at	DATETIME	DEFAULT utcnow	

Table: questions

Column	Type	Constraints	Notes
id	UUID	PK	
domain_id	UUID	FK -> domains.id, NOT NULL	Owning domain
question_text	TEXT	NOT NULL	English
question_text_ar	TEXT	NOT NULL	Arabic
question_order	VARCHAR(10)	NOT NULL	Display order
is_active	BOOLEAN	DEFAULT TRUE	Soft delete

Table: user_sessions

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	FK -> users.id, NOT NULL	Owner
domain_id	UUID	FK -> domains.id, NOT NULL	Selected domain
country	VARCHAR(10)	NOT NULL	e.g. JO

role	VARCHAR(50)	NOT NULL, DEFAULT 'busine: Quadrant role	
answers	TEXT	NULLABLE	JSON serialised Q&A
created_at	DATETIME	DEFAULT utcnow	

Table: requirements

Column	Type	Constraints	Notes
id	UUID	PK	
session_id	UUID	FK -> user_sessions.id, NOT NULL	Parent session
code	VARCHAR(20)	NOT NULL	FR-1, NFR-2 ...
description	TEXT	NOT NULL	AI or edited text
type	VARCHAR(20)	NOT NULL	functional non_functional
is_edited	BOOLEAN	DEFAULT FALSE	User-modified flag
created_at	DATETIME	DEFAULT utcnow	
updated_at	DATETIME	DEFAULT utcnow	Updated on PUT

Table: audit_logs

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	FK -> users.id ON DELETE SE	Nullable (anonymous)
action	VARCHAR(100)	NOT NULL	e.g. domain_created
entity_type	VARCHAR(50)	NOT NULL	domain, user, question
entity_id	VARCHAR(255)	NULLABLE	UUID of affected row
details	JSON	NULLABLE	Arbitrary metadata
ip_address	VARCHAR(45)	NULLABLE	IPv4/IPv6
created_at	DATETIME	DEFAULT utcnow, INDEX	

Indexes: ix_audit_logs_user_id, ix_audit_logs_created_at, ix_audit_logs_action

Table: login_history

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	FK -> users.id ON DELETE SE	Nullable
email_attempted	VARCHAR(255)	NOT NULL, INDEX	Attempted email
success	BOOLEAN	NOT NULL	
ip_address	VARCHAR(45)	NULLABLE	
user_agent	VARCHAR(500)	NULLABLE	Browser string
failure_reason	VARCHAR(50)	NULLABLE	user_not_found wrong_password account_disa

created_at

DATETIME

DEFAULT utcnow, INDEX

Indexes: ix_login_history_email, ix_login_history_created_at, ix_login_history_user_id

Entity Relationships (ER Summary)

```

users (1) ----< (N) refresh_tokens      [user_id]
users (1) ----< (N) password_reset_tokens [user_id]
users (1) ----< (N) user_sessions       [user_id]
users (1) ----< (N) audit_logs          [user_id, ON DELETE SET NULL]
users (1) ----< (N) login_history       [user_id, ON DELETE SET NULL]
domains(1) ----< (N) questions          [domain_id]
domains(1) ----< (N) user_sessions      [domain_id]
user_sessions(1) ---< (N) requirements  [session_id]

```

Notable Database Design Decisions

- UUIDs as PKs: Prevents enumeration attacks and simplifies multi-instance inserts.
- SHA-256 token hashing: Refresh/reset tokens stored as hashes - even if the DB is compromised, tokens cannot be replayed without the original string.
- ON DELETE SET NULL on audit/login FKs: Deleting a user preserves audit history - user_id becomes NULL but the log entry survives.
- Soft deletes (is_active): Domains and questions are never hard-deleted; is_active=False hides them from users but preserves historical data.
- answers as TEXT (JSON): Flexible - no schema migration needed when question sets change. Trade-off: no SQL-level querying of individual answers.
- No explicit SQLAlchemy relationship() declarations: FK constraints exist in migrations, but ORM relationships are not declared - queries join manually. Simple but limits lazy-loading conveniences.

7 Sessions - Authentication & Application Sessions

The project uses two distinct concepts both called 'session', which is important to distinguish:

Type A: JWT Authentication Sessions

This is the standard HTTP authentication mechanism. It consists of two tokens:

A1 - Access Token (Short-lived JWT)

Created by `create_access_token()` in `security.py`. Signed with HS256 using `SECRET_KEY`. Payload: `{'sub': user_id, 'exp': utcnow + ACCESS_TOKEN_EXPIRE_MINUTES}`. Lifetime configured in `.env` (typically 15-60 minutes). NOT stored in the database - stateless. Sent in every API request as: Authorization: Bearer <token>. Verified by `verify_token()` which decodes the JWT and checks expiry.

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({'exp': expire})
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
```

A2 - Refresh Token (Long-lived JWT, DB-backed)

Created by `create_refresh_token()`. Same structure as access token but with a longer lifetime (configured as `REFRESH_TOKEN_EXPIRE_DAYS`, typically 7 days). The raw JWT is returned to the frontend (stored in `localStorage`). The SHA-256 hash of the token is stored in the `refresh_tokens` table. On `/auth/refresh`, the backend: (1) decodes the JWT to check signature/expiry, (2) computes SHA-256 hash and looks it up in DB, (3) checks `is_revoked=False`, (4) issues a new access token. On `/auth/logout`, the hash is found and `is_revoked` is set to `True`.

```
# On logout - revoke the refresh token
token_hash = hashlib.sha256(data.refresh_token.encode()).hexdigest()
db_token = db.query(RefreshToken).filter(RefreshToken.token_hash == token_hash).first()
if db_token:
    db_token.is_revoked = True
    db.commit()
```

A3 - Password Reset Token (One-Time Use)

Generated with `secrets.token_urlsafe(32)` - a cryptographically secure random URL-safe string. The SHA-256 hash is stored in `password_reset_tokens` with:

```
expires_at = utcnow + 1 hour
is_used = False
```

The raw token is embedded in the reset link emailed to the user. On use: the hash is looked up, expiry checked, `is_used` set `True`, and password updated. Anti-enumeration: the forgot-password endpoint always returns the same message regardless of whether the email exists.

Frontend Token Storage

Tokens are stored in browser `localStorage` (not `HttpOnly` cookies). This is a common React/SPA pattern but has a security trade-off: `localStorage` is accessible to JavaScript, so XSS vulnerabilities could expose tokens. For higher security, migrate to `HttpOnly` cookie storage with `SameSite=Strict`.

Security note: The current localStorage approach is acceptable for development/MVP.
For production with sensitive data, consider HttpOnly cookies + CSRF tokens instead.

Type B: Application / Questionnaire Sessions (user_sessions table)

This is a completely separate concept from authentication. A 'user session' here represents one requirements-generation workflow - a snapshot of a user's choices at a point in time.

When a user completes the questionnaire and clicks 'Generate Requirements', a POST to /domains/session creates a user_sessions row containing:

- user_id: who started this session
- domain_id: which domain was selected (Health, Education, Finance)
- country: typically 'JO' for Jordan
- role: the quadrant role (product_owner, business_analyst, developer, stakeholder)
- answers: JSON string of [{question_id, answer}, ...]
- created_at: timestamp

This session_id is then the primary key linking all downstream data: requirements, cross-check results, SRS generation, and use-case generation all query by session_id.

Session Lifecycle Diagram

User Answers Questionnaire

|
v

POST /domains/session -> user_sessions row created -> session_id returned

|

+---> POST /requirements/generate (session_id) -> requirements rows saved

|

+---> GET /crosscheck/{session_id} <---+ (reads requirements)

|

+---> POST /srs/generate (session_id) <---+ (reads requirements)

|

+---> POST /usecases/generate (session_id) <---+ (reads FR requirements)

Summary: Two Types of Session Compared

Attribute	Auth Session (JWT)	App Session (user_sessions)
Purpose	Authenticate API calls	Track requirements workflow
Storage	localStorage (client) + DB hash	PostgreSQL table
Lifetime	15min access / 7day refresh	Permanent (no expiry)
Created at	Login / Register	Questionnaire completion

Invalidation	Logout (revoke) or expiry	Never deleted (historical record)
Key field	JWT 'sub' claim = user UUID	session_id UUID

End of Report

RequirementsSuperTool | Project Analysis | May 2026

This document was auto-generated by analysing the full source code of the project, including all backend Python modules, SQLAlchemy models, Alembic migrations, FastAPI routers, Pydantic schemas, and Next.js/TypeScript frontend files.